

Necessarily Rank-Maximal Matching

Madhav P Nair and Devapriya Pradeep

September 2025

Contents

1	Introduction	3
1.1	Previous and Related works	4
1.2	Outline of the Thesis	5
2	Preliminaries	6
2.1	Basic Definitions and Notations	6
2.2	Preliminaries for Rank-Maximal Matching	7
2.3	Preliminaries for Necessarily Rank-Maximal Matching	8
3	Maximum Matching	11
3.1	Definition	11
3.2	Naive Algorithm for finding Maximum Matching using Augmenting Paths	12
3.3	$O(n^3)$ Algorithm for Finding Maximum Matching in Bipartite Graphs	12
3.3.1	Time complexity analysis:	13
4	Rank-Maximal Matching	15
4.1	Definition	15
4.2	Combinatorial Algorithm to compute Rank-Maximal Matching - $O(\min(C\sqrt{n}, n + C)m)$	16
4.2.1	Plan	17
4.2.2	Gallai-Edmonds Decomposition	17
4.2.3	The Algorithm	18
4.2.4	Time Complexity Analysis	19
5	Necessarily Rank-Maximal Matching	21
5.1	Elicitation Protocols (Query Models)	21
5.1.1	Next-best query model	21
5.1.2	Hybrid-query model	22
5.1.3	Set-compare model	22
5.2	Checking whether a matching is NRM	22
5.2.1	Helper Algorithm to find optimal matching	22

	Time Complexity Analysis	23
5.2.2	Algorithm to determine whether a matching is NRM . . .	23
	Time Complexity Analysis	24
5.3	Computing an NRM, if it exists	24
	Time Complexity Analysis	25
5.4	Elicitation Algorithm for Necessarily Rank-Maximal Matching with Next-Best Model	26
	Competitive Ratio	27
6	Our Results	28
6.1	Limitations of Set-Compare model	28
6.2	Modifying Set-Compare model	30
6.3	Rank-Aware Set-Compare Model	31
6.4	Analysis of the Algorithm	36

Chapter 1

Introduction

Matching with preferences, though a niche topic of study in Graph Theory, has rather widespread applications in today's world. Matching students to their preferred projects, hostellers to their preferred rooms are some examples of such matching problems that we may come across in our daily life. Formally, given a set of agents and objects, a matching is just a pairing of agents to objects where only one agent can be paired with one object and vice versa. This report focuses only on one-sided matchings, where only agents have preferences over objects, though we can have matchings where both sides have preferences as well. We wish to make this matching as fair as possible with regard to each agent's preferences.

It is often not possible to match every agent with their first choice without violating the property of a matching. Therefore, a realistic goal is to keep as many agents as satisfied as we can. Several optimality criteria have been studied in the literature, such as Pareto optimal, rank-maximal, fair, and popular matchings [9, 1, 11]. In this project, we focus on the Rank-Maximal Matching (RMM) criterion.

The input for this problem consists of a set of agents A and a set of objects P . Each agent $a \in A$ has a preference list over the set of objects P . For any pair (a, p) with $p \in P$, the edge (a, p) is said to have rank i if p is the i th choice of agent a . The output is a matching M , that matches the maximum number of agents to their rank-1 objects, and subject to this, the maximum number to their rank-2 objects, and so on.

A rank-maximal matching can be computed in $O(\min(c\sqrt{n}, n)m)$ time, where n is the number of vertices, m is the number of edges, and c is the maximum rank of an edge in an optimal solution. However, in some scenarios it may be necessary to provide the full preference list for each agent. This may become very inconvenient when the number of agents or objects is large. For example, in a project allocation setting, asking each student in a class of 100 students to rank all 100 available projects would be inefficient, if everyone is

matched to one of their top 10 choices.

This leads us to search for a necessarily rank-maximal matching, which requires only a partial list of preferences from each agent but still searches for a rank-maximal matching in the revealed preference list such that it would remain the optimal rank-maximal matching for the system, irrespective of the values of the unrevealed preferences. This means that we just need to query our agents for their first few most preferred items (the number of queries can differ for each agent), and a necessarily rank-maximal matching would still give us the best possible allocation, saving us the time and work of a large number of queries!

Since we do not know how many minimum queries we need exactly for each agent to get a necessarily rank-maximal matching, we study an online elicitation algorithm in which we start with empty preference lists for each agent and ask queries till all the agents are optimally matched. This saves us the work of asking queries that are unnecessary!

1.1 Previous and Related works

The study of matchings under preferences began with the seminal work of Gale and Shapley [4], who introduced the Stable Marriage Problem and the Deferred Acceptance (DA) algorithm. Their model considered two-sided preferences, where two distinct sets of agents rank one another, and the goal is to find a stable matching. This led to extensive research on two-sided preferences, including extensions to many-to-one settings, such as the hospital–resident problem [14], and algorithms that handle incomplete or tied preferences [8, 6].

Another line of work considers settings where only one side of the bipartition has preference lists. Several optimality criteria have been studied in the literature. Some of the well-known optimality criteria in this context include rank-maximality [9], popularity [1], and fairness [11]. More recently, researchers have begun exploring robustness and necessity in settings where preferences are partial or uncertain. A necessarily stable matching [15, 2] remains stable under all possible completions of incomplete preferences. This concept has been extended to one-sided models, resulting in necessarily rank-maximal and necessarily popular matchings.

Another direction examines preference elicitation, namely, how much preference information must be collected to guarantee robust outcomes. Hosseini et al. [7] introduced the notions of necessarily Pareto-optimal (NPO) and necessarily rank-maximal (NRM) matchings (i.e., matchings that maintain optimality across all completions) and provided algorithms for checking existence as well as eliciting partial preferences in an online query model. Building on this, Peters [?] proposed several query models. Similarly, Bampis et al. [3] studied online query-minimization strategies for stable matchings under one-sided uncertainty, designing algorithms that maintain stability while minimizing the number of

preference queries.

1.2 Outline of the Thesis

This work mainly focuses on exploring rank-maximal matchings which includes presenting the formal definitions and some of the algorithms and its time complexity analysis. The following chapters go through the very prerequisites to do so. First of all we will see some basic definitions and notations in the Chapter 2 and then advance to Maximum matchings. This chapter shows a general algorithm for computing maximum matchings followed by a $O(n^3)$ algorithm in case of bipartite matchings. These serve as some essentials for the combinatorial algorithm discussed in Chapter 4 to produce rank-maximal matchings in a bipartite graph. First we discuss about the Gallai–Edmonds decomposition where we partition the vertices into three pairwise disjoint subsets. Next we present the combinatorial algorithm for computing rank-maximal matching. We conclude by presenting correctness proof and the running time analysis of the algorithm. Chapter 5 provides a detailed literature survey on necessarily rank-maximal (NRM) matchings. Finally, Chapter 6 presents our contributions. We first highlight the drawbacks of the regular set-compare model used for analysing NRM matchings. Motivated by these limitations, we introduce a modified set-compare model that addresses these issues. We then present our online algorithm for this revised model and conjecture that it achieves a constant competitive ratio, improving upon the currently known linear upper bound.

Chapter 2

Preliminaries

2.1 Basic Definitions and Notations

Here we look into some basic definitions and notations which will be helpful for further reading.

As we know, a **graph** $G = (V, E)$ is a collection of **vertices** (or points) V and **edges** E , where each edge (for example, (v_1, v_2) with $v_1, v_2 \in V$) connects a pair of vertices to show how they are linked or related.

First of all, let us check the definition of a **matching** and some related ideas.

Definition 2.1.1 (Maximum Matching). A subset of the edge set of a graph is called a **matching** if no two edges in the set share an endpoint. Let $G = (V, E)$ be a graph. A subset $M \subseteq E$ is called a matching of G if

$$\forall v \in V, \quad |\{e \in M : v \text{ is incident on } e\}| \leq 1.$$

A matching M of G is said to be **maximum** if for every matching M_0 of G , we have $|M| \geq |M_0|$.

Definition 2.1.2 (Maximal Matching). A matching M of G is said to be **maximal** if for every $e \in E \setminus M$, the set of edges $M \cup \{e\}$ is not a matching of G .

Definition 2.1.3. The **size** of a matching M of G is the number of edges it contains and is denoted by $|M|$.

Corollary 2.1.1. A maximum matching is always maximal, but not vice versa.

Definition 2.1.4 (M-saturated vertex). Let M be a matching of G . A vertex $v \in V$ is said to be **M -saturated** (or **matched** with respect to M) if M contains an edge incident on v . Otherwise, v is said to be **M -unsaturated** (or **free** with respect to M).

Definition 2.1.5 (Perfect Matching). A matching M of G is said to be **perfect** if all vertices of G are M -saturated.

Definition 2.1.6 (Alternating Path). A path P in G is said to be M -**alternating** if the edges of P alternate with respect to membership in M .

Definition 2.1.7 (Augmenting Path). A path P in G is said to be M -**augmenting** if P is a maximal M -alternating path that starts and ends at M -unsaturated vertices

Corollary 2.1.2. Every M -augmenting path must have odd length.

Let P be an augmenting path with respect to M . Then

$$M' \leftarrow M \oplus P$$

denotes the matching obtained by **augmenting** M using P , which updates M to a larger matching M' .

2.2 Preliminaries for Rank-Maximal Matching

Now let us introduce some technical ideas required for the Rank-Maximal Matching problem.

Let

$$N = \{a_1, a_2, \dots, a_n\}$$

denote the set of **agents** who have to be matched, and let

$$O = \{o_1, o_2, \dots, o_n\}$$

denote the set of **objects** to which the agents are to be matched.

Definition 2.2.1 (Preference Profile). P_i denotes the **preference list** of agent a_i , which ranks the objects according to her preference. The collection of all preference lists is called the Preference Profile and denoted by

$$P = (P_1, P_2, \dots, P_n).$$

We define a bipartite graph

$$G = (N \cup O, E),$$

where the edge set E contains all pairs (a, o) such that object o appears in the preference list of agent a .

Definition 2.2.2 (Signature). Let r be the largest rank that any agent uses to rank an object. The **signature** of a matching M is defined to be the r -tuple

$$(x_1, x_2, \dots, x_r),$$

where x_i is the number of agents who are matched in M with one of their i^{th} choice objects.

Each edge $(a, o) \in E$ is labeled according to the rank of object o in the preference list of agent a . Thus,

$$E = E_1 \cup E_2 \cup \dots \cup E_r,$$

where E_i denotes the set of edges of **rank** i .

Therefore, our bipartite graph G can be represented as

$$G = (N \cup O, E_1 \cup E_2 \cup \dots \cup E_r).$$

We use $[k]$ to denote the set $\{1, 2, \dots, k\}$. Thus, for $i \in [n]$, a_i and o_i represent agent i and object i , respectively. A matching of agents to objects is a **bijection**

$$M : N \rightarrow O.$$

We may also refer to a matching as a collection of pairs

$$\{(a_i, M(a_i)) \mid i \in [n]\}.$$

Finally, let M denote the set of all possible matchings in G .

2.3 Preliminaries for Necessarily Rank-Maximal Matching

We have to start with Partial preference lists to study NRMs.

Definition 2.3.1 (complete preference profile). A complete preference profile $R = (R_1, R_2, \dots, R_n)$ where each R_i , $i \in [n]$ is the preference order of agent i on all n objects.

This is what we saw in Definition 2.2.1 which is to be used in the offline algorithm given in 4.2 where all the preferences should be revealed.

Definition 2.3.2 (partial preference profile (top- k preferences)). A partial preference profile $P = (P_1, P_2, \dots, P_n)$ where each P_i , $i \in [n]$ is a prefix of the complete preference order of agent i on k_i objects. Each agent i reveals top k_i preferences in P . k_i can be different for each agent.

Definition 2.3.3 ($\text{rev}(P)$). It is the set of all pairs (a_i, o_j) such that a_i has revealed object o_j in P_i .

Definition 2.3.4 ($\text{unrev}(P)$). $\text{unrev}(P) = N \times O - \text{rev}(P)$ is simply the set of all agent-object pairs for which the rank is unrevealed in the preference list.

Definition 2.3.5 (size of M w.r.t P). size of a matching M w.r.t a given preference profile P is given by $|M \cap \text{rev}(P)|$

Definition 2.3.6 (Consistency of R). A complete preference list R_i is said to be *consistent* with a partial preference list P_i iff the relative preference of any two pair of objects is held same as in P_i
 $\forall j, k \in [n], o_j \succ_{P_i} o_k \implies o_j \succ_{R_i} o_k$
The set of all R_i consistent with P_i is denoted as $C(P_i)$. Similarly, $C(P)$ gives the set of all R consistent with P .

Now let us familiarize with some type of signatures which are crucial in NRM.

Recall Definition 2.2.2 of signature. Here we define signature but in the context of partial preferences.

Definition 2.3.7 (Signature). Given any top-k preference profile, the *signature* of M w.r.t P , denoted by $sig_P(M)$ is the n-tuple (where n is the number of agents(objects)) $(x_1, x_2, \dots, x_n)$ where x_i denotes the number of agents who get matched to their i^{th} preference exactly as defined earlier.
The key point is that no unrevealed pairs are considered while computing $sig_P(M)$. It is the signature of revealed pairs only.

Definition 2.3.8 (rank-maximality w.r.t top-k preferences). We say that $(x_1, \dots, x_n) \succ_{sig} (x'_1, \dots, x'_n)$ if there exists an $h \in [n]$ such that $x_h > x'_h$ and $x_i = x'_i \forall i < h$; this is the lexicographic comparison of signatures. A matching M is rank-maximal w.r.t. P if $sig_P(M)$ is maximum under $\succ_{sig}$.

Definition 2.3.9 (Extended Signature). Extended signature of a matching M w.r.t P denoted $extsig_P(M)$ is the tuple $(x_1, \dots, x_n)$ where all $i \in [n-1]$, x_i is defined as in $sig_P(M)$ and the number of agents matched to unrevealed objects (in their preference list) are counted in x_n along with actual n^{th} preference matchings.

This can be interpreted as a matching with worst $sig_R(M)$ among matchings with same $sig_P(M)$ since we assume all unrevealed pairs as the least preferred events.

Definition 2.3.10 (Optimal Signature). Given a top-k preference profile P , we define $sig_P^{opt}(M)$ as the *best possible signature* that any matching can achieve under any completion of P .

Let $S \subseteq N$, $T \subseteq O$ and $F \subseteq S \times T$ be the forbidden pairs and $\mathcal{M}(S, T, F)$ be the set of all matchings M that match agents in S to T and satisfy $M \cap F = \emptyset$.

$$sig_P^{opt}(S, T, F) = \sup_{\mathcal{M}(S, T, F), R \in C(P)} sig_R(M)$$

By default, sig_P^{opt} means $sig_P^{opt}(N, O, \emptyset)$.

Online algorithms elicit preferences of agents from an empty preference profile using different query methods. We are interested in measuring the performance of the online algorithm by comparing it with all-powerful hypothetical offline optimal algorithm.

Definition 2.3.11 (Competitive ratio). We say an online elicitation algorithm is α -competitive or has competitive ratio α if the number of queries it requires to ask in the worst-case across all possible instances is at most $\alpha \cdot \text{OPT}$ where OPT is the number of queries asked by the optimal algorithm. The optimal algorithm knows the minimum number of queries sufficient to guarantee an NRM matching.

Chapter 3

Maximum Matching

3.1 Definition

To find maximum matchings in graphs, we shift our perspective from viewing matchings as a set of pairings to viewing them as paths made of alternating edges. In other words, we take a path in the original graph such that the edges of the path alternate between being a matched edge and not being a matched edge. [12] Such paths are called alternating paths. Specifically, **M -alternating path**, if M is the matching involved.

Definition 3.1.1. A path P in G is referred to as M -alternating if its edges alternate between being matched and unmatched with respect to a matching M .

Any alternating path that is maximal and starts and ends with unmatched vertices in M is also termed an **M -augmenting path**.

Definition 3.1.2. A path P in G is referred to as M -augmenting if it is a maximal M -augmenting path that start and end at free vertices.

The utilization of these terms appears in the **Berge lemma** (1956).

Lemma 1. *A matching M in G is maximum if and only if G does not have an M -augmenting path.*

This is because if there was an M -augmenting path for a maximum matching, we could just flip (or augment) the status of the edges of the path with respect to its membership in M . Clearly, this would match one more edge compared to the original path. Matching one more edge would imply that the size of the matching is greater by one, which means that our newly obtained matching has a size larger than the matching we originally thought to be maximum. This creates a contradiction and therefore convinces us that for a maximum matching M , we would not be able to find a M -augmenting path.

3.2 Naive Algorithm for finding Maximum Matching using Augmenting Paths

Berge Lemma gives an intuitive and naive algorithm for finding the maximum matching. We start with an empty matching, check for M -augmenting paths, and on finding one, we flip it to get a better matching. We continue doing this until no M -augmenting path can be found in the graph. The pseudo code of the same is given below:

Algorithm 1 Maximum Matching

```

1:  $M = \emptyset$ 
2: while (there exists an  $M$ -augmenting path  $P$ ) do
3:    $M = M \oplus P$             $\triangleright$  augmenting the path to get a better matching
4: end while
5: return  $M$ 

```

3.3 $O(n^3)$ Algorithm for Finding Maximum Matching in Bipartite Graphs

We modify the above algorithm to find maximum matching in bipartite graphs. Since we want the matching edges to be cross edges with respect to sets A and B , we can see that augmenting paths here would start from any free vertex in B (i.e., a vertex in B_o), and would have alternating unmatched and matched edges until it ends up in a vertex in A (i.e. a vertex in A_o). To make this statement clearer, we create a graph H such that it has the same vertices as G . H has an edge (a, b) from A to B if (a, b) is part of the matching M . Otherwise, H has (b, a) as an edge.

Clearly, we see that if an M -augmenting path P from $u \in A$ to $v \in B$ exists in G , H would trace the same path in a zigzag manner starting from $v \in B$ until it reaches $u \in A$, and vice versa.

We start the algorithm with an empty matching, just like the previous naive algorithm. Since the constructed graph H depends on M , H needs to be updated as M is updated, also. So, like in the previous algorithm, we iterate as long as we find an M -augmenting path in G , and continuously update M . When there is no longer any M -augmenting path in G , it follows from Berge's lemma that M is the maximum matching. In each iteration, we have to start DFS from each vertex of B_o to find an M -augmenting path. Since this is cumbersome and may require multiple DFS traversals, we add an additional vertex β to H and add edges from β to all vertices in B_o so that a single DFS traversal from β would be sufficient to find an M -augmenting path. When we find an M -augmenting path in G , we augment it and update M as the same.

After finding a better matching, we update H again using the new matching and proceed with the next steps of the algorithm to get an even better matching. This is repeated as long as we find an M -augmenting path in G . When that is no longer the case, we return M , which turns out to be the maximum matching.

The pseudo code for the same is provided below for a bipartite graph $G = (A \cup B, E)$.

Algorithm 2 Maximum Matching

```

1:  $M = \emptyset$ 
2: while there exists an  $M$ -augmenting path  $P$  in  $G$  do
3:   Construct a new graph  $H$ 
4:    $V(H) = V(G)$ 
5:    $A_o \leftarrow$  set of unmatched vertices in  $A$ 
6:    $B_o \leftarrow$  set of unmatched vertices in  $B$ 
7:   for each  $(a, b) \in E(G)$  do
8:     if  $(a, b) \in M$  then
9:       Add  $(a, b)$  to  $E(H)$ 
10:    else
11:      Add  $(b, a)$  to  $E(H)$ 
12:    end if
13:  end for
14:  Add vertex  $\beta$  to  $V(H)$ 
15:  for each  $b \in B_o$  do
16:    Add  $(\beta, b)$  to  $E(H)$ 
17:  end for
18:  Start DFS in  $H$  from  $\beta$ 
19:  if DFS ends at  $a \in A_o$  then
20:     $P =$  Path in  $G$  corresponding to the path found by DFS in  $H$ 
21:     $M = M \oplus P$ 
22:  end if
23: end while
24: return  $M$ 

```

3.3.1 Time complexity analysis:

Each iteration of the outer loop augments the matching M , and thereby increases the size of the matching by 1. In other words, it matches one free vertex of B during one iteration. Since B has at most $n/2$ vertices, the outer loop would run for at most $n/2$ iterations, which can be completed in $O(n)$ time.

The construction of H takes $O(n + m)$ time, since all the vertices and edges must be copied to H from G . The addition of β takes $O(n)$ time, since β can be connected to most of the n vertices in H . DFS takes $O(n + m)$ time. Augmentation of the matching M along the path P takes $O(m)$ time, since the

status of each edge on the path must be toggled.

Therefore, the algorithm runs in $O(\frac{n}{2} \times (n + m))$ time = $O(n^3)$ time as m can be at most $n^2/4$ in a bipartite graph.

Chapter 4

Rank-Maximal Matching

4.1 Definition

In Section 2, we discussed the preference lists of each agent, the ranks assigned by the agents to the objects, and the signature of a matching M . Recall that the signature of a matching is an r -tuple $(x_1, x_2, \dots, x_r)$. We can abbreviate it as $(x_1, x_2, \dots, x_d)$ if $x_d > 0$ and $x_i = 0$ for all $i = d + 1, \dots, r$. (note that r is the highest rank used in the matching)

Definition 4.1.1 (Rank-Maximal Matching). A **rank-maximal matching** is a matching that has the *maximum signature* (in lexicographic order) among all possible matchings.

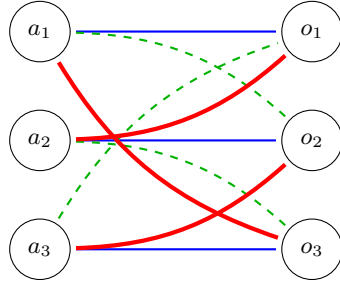
Eg: if a matching M_1 has signature $(2,1,0)$ and another matching M_2 has signature $(1,2,0)$ it means under M_1 two agents got their rank 1 object and one agent got rank 2 and none of them were allocated with objects having 3 or higher ranks. M_2 maps only a single agent to its most preferred object and two agents to the second item in their preference lists and no one got 3 or higher ranks. Clearly, M_1 has greater signature (lexicographical order) than M_2 .

Let us consider an example to get a clear notion of a rank-maximal matching. Suppose the preference profile P is

$$\begin{aligned}a_1 : o_1 \succ o_2 \succ o_3, \\a_2 : o_2 \succ o_3 \succ o_1, \\a_3 : o_3 \succ o_1 \succ o_2.\end{aligned}$$

In Figure 4.1, three matchings are shown and let us figure out the signature of each matching and where the rank-maximality is obtained.

$\mathcal{M}$ be the set of all matchings from N to O . $\mathcal{M}_1$ is the set of all matchings in which the maximum possible number of agents are matched to their first choice.



$$\begin{aligned}
M_1 &= \{(a_1, o_1), (a_2, o_2), (a_3, o_3)\}, & (\text{blue}), & \text{sig}(M_1) = (3, 0, 0), \\
M_2 &= \{(a_1, o_2), (a_2, o_3), (a_3, o_1)\}, & (\text{green}), & \text{sig}(M_2) = (0, 3, 0), \\
M_3 &= \{(a_1, o_3), (a_2, o_1), (a_3, o_2)\}, & (\text{red}), & \text{sig}(M_3) = (0, 0, 3).
\end{aligned}$$

Comparing the signatures, $\text{sig}(M_1) \succ \text{sig}(M_2) \succ \text{sig}(M_3)$.

Clearly, M_1 is the rank-maximal matching.

Figure 4.1: Three matchings: M_1 (blue), M_2 (green dashed), M_3 (red).

$\mathcal{M}_2$ is the set of all matchings in which the maximum possible number of agents are matched to their second choice subject to the condition that the maximum possible first rank matchings are already taken.

So, $\mathcal{M}_2 \subseteq \mathcal{M}_1$

Similarly, $\mathcal{M}_i \subseteq \mathcal{M}_{i-1}$

$\mathcal{M}_r$ is the set of all rank-maximal matchings and

$\mathcal{M}_r \subseteq \mathcal{M}_{r-1} \dots \subseteq \mathcal{M}_i \subseteq \mathcal{M}_{i-1} \dots \mathcal{M}_2 \subseteq \mathcal{M}_1$

4.2 Combinatorial Algorithm to compute Rank-Maximal Matching - $O(\min(C\sqrt{n}, n + C)m)$

Now we have a combinatorial algorithm for computing a rank-maximal matching in a bipartite graph $G = (N \cup O, E)$ given by Irving[10].

Here we assume that no two objects have the same rank for any agent (no ties between objects for a given rank in the preference list of an agent) in N , i.e. only one edge from E_i is incident on an agent a .

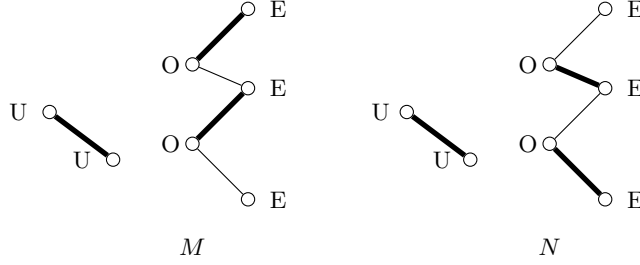


Figure 4.2: M and N are maximum matchings. Thick edges are matched and thin are unmatched.

4.2.1 Plan

The core idea of this algorithm is to **convert the rank-maximal matching problem to a maximum matching problem in a reduced graph**. The reduced graph is obtained by pruning the graph in each step deleting the edges that will never belong to a rank-maximal matching that we will see in section 4.2.3.

4.2.2 Gallai-Edmonds Decomposition

In matching theory, it is a common practice to structurally decompose a graph into Odd, Even and Unreachable vertices to study the structure of maximum matchings.

Given a graph $G = (V, E)$ and any maximum matching M of G , we define three sets as follows:

- **Even ($\mathcal{E}$) vertices:** Vertices that can be reached from a free vertex by an even-length alternating path w.r.t. M .
- **Odd ($\mathcal{O}$) vertices:** Vertices that can be reached from a free vertex by an odd-length alternating path w.r.t. M .
- **Unreachable ($\mathcal{U}$) vertices:** Vertices that cannot be reached from any free vertex by any M -alternating path.

In Figure 4.2.2, both M and N are two different maximum matchings. We can observe that for any maximum matching:

- The sets $\mathcal{E}$, $\mathcal{O}$, and $\mathcal{U}$ are **disjoint** and **independent** of the particular matching M that defines them, since both M and N yield the same $\mathcal{E}$, $\mathcal{O}$, and $\mathcal{U}$.
- The subgraph $G' = (V, M) \text{ or } (V, N)$ contains only $\mathcal{U}-\mathcal{U}$ and $\mathcal{O}-\mathcal{E}$ edges.
- Every vertex in $\mathcal{O}$ (through $\mathcal{O}-\mathcal{E}$ edges) and every vertex in $\mathcal{U}$ (through $\mathcal{U}-\mathcal{U}$ edges) is **matched**.

- The **cardinality** of a maximum matching is given by

$$|\mathcal{M}| = |\mathcal{O}| + \frac{|\mathcal{U}|}{2}.$$

- There is **no** $\mathcal{E}$ - $\mathcal{U}$ edge and no $\mathcal{E}$ - $\mathcal{E}$ edge in G' .

4.2.3 The Algorithm

Let us look into an algorithm for constructing a **rank-maximal matching** M .

Let $G_i = (N \cup O, E_1 \cup \dots \cup E_i)$ and G'_i is the pruned graph at stage i (as discussed in section 4.2.1).

Right now, we can check how the graph is reduced at each iteration and what all edges are kept or deleted based on the above observations in order to find a maximum matching in the graph G_i at each stage i which serves as a rank-maximal matching in graph G as if i is the maximum rank.

Initially, $G'_1 = G_1$, and let M_1 be any maximum matching in G'_1 . For $i = 1$ to $r - 1$, perform the following steps, and output M_r which is nothing but a maximum matching in the r^{th} iteration of the algorithm.

Algorithm 3 Elicitation Procedure for NRM

- 1: **for** $i = 0$ to $n - 1$ **do**
 - 2: **Step 1:** Partition the nodes of $N \cup O$ into three disjoint sets $\mathcal{E}_i$, $\mathcal{O}_i$, and $\mathcal{U}_i$ using an alternating BFS (i.e., by growing a Hungarian forest[†]).
 - 3: **Step 2:**
 - Delete all edges incident to nodes in $\mathcal{O}_i \cup \mathcal{U}_i$ from each $\mathcal{E}_j$ for all $j > i$.
 - Remove all $\mathcal{O}_i \mathcal{O}_i$ and $\mathcal{U}_i \mathcal{U}_i$ edges from G_i .
 - Add the edges in E_{i+1} to G'_i to obtain the updated graph G'_{i+1} .
 - 4: **Step 3:** Compute a maximum matching M_{i+1} in G'_{i+1} by augmenting M_i .
 - 5: **end for**
-

4.2.3.1 [†] Alternating BFS for Hungarian Forest

Alternating BFS is a way to explore all the M-alternating paths starting from every free vertex (Multi-source start).

Start with all free (unmatched) vertices as sources $\rightarrow$ these are even (distance 0). They're the roots of the Hungarian forest.

From each even vertex:

Traverse all unmatched edges to reach new vertices $\rightarrow$ these become odd.
 From each odd vertex:
 Follow its matched edge (the unique one in M) $\rightarrow$ reach a new even vertex.
 Continue until no new vertex is discovered.
 Finally:
 $\mathcal{E}_i$: vertices reached at even depth $\rightarrow$ even vertices.
 $\mathcal{O}_i$: vertices never reached at odd depth $\rightarrow$ odd vertices
 $\mathcal{U}_i$: $V - (\mathcal{E} \cup \mathcal{O}) \rightarrow$ unreachable vertices.

4.2.3.2 Pruning

Here, in step 2 of the algorithm, we delete all edges that have a higher rank than i from E that are incident to a node in $\mathcal{O} \cup \mathcal{U}$ and all $\mathcal{O}_i \mathcal{O}_i$ and $\mathcal{O}_i \mathcal{U}_i$ edges during every i th iteration.

From Section 4.2.2, it is clear that all vertices of $\mathcal{O}_i$ and $\mathcal{U}_i$ are already matched in any maximum matching in G_i . So, $\mathcal{O}_i \cup \mathcal{U}_i$ contains only saturated vertices and adding higher-rank edges incident to them cannot produce a better matching but only worsens the signature. So, avoid any higher ranks to these vertices since they spoil the rank-maximality of the final matching.

Also, all $\mathcal{O}_i \mathcal{O}_i$ and $\mathcal{U}_i \mathcal{U}_i$ edges are pruned because they never appear in a maximum matching since they connect already matched vertices and are safely deleted.

4.2.4 Time Complexity Analysis

In any iteration i , we first determine the partition of the vertices into $\mathcal{E}_i$, $\mathcal{O}_i$, and $\mathcal{U}_i$. This can be performed in $O(m)$ time by growing a Hungarian forest with respect to the matching M_i .

Once the partition is obtained, the algorithm removes the appropriate edges from the edge set, again requiring $O(m)$ time.

Next, we compute M_{i+1} from M_i by augmenting along the augmenting paths. Using the Hopcroft-Karp algorithm [12], this step takes $O(\min(\sqrt{n}, |M_{i+1}| - |M_i| + 1) \cdot m)$ time. Since there are r iterations in total, the overall running time of the algorithm is $O(\min(r\sqrt{n}, n+r) \cdot m)$. This follows from the following observations:

$$\sum_{i=1}^r \sqrt{n} = r\sqrt{n}$$

$$\sum_{i=1}^r |M_{i+1} - M_i| = M_r - M_0 + r = n + r.$$

To further optimize, we can replace r with C , where it is important to note the distinction that r is the highest rank that appears in the preference list of an agent and C denotes the highest rank that appears in the final rank-maximal matching. At the beginning of each iteration i , we first check whether M_i is

already a maximum matching in G'_r , where G'_r is the graph containing all edges (of all ranks) that remain at the start of iteration i . This check can be performed in $O(m)$ time.

If M_i is a maximum matching in G'_r , the algorithm terminates; otherwise, there exists an edge of rank greater than i whose addition increases the cardinality of the maximum matching, implying $C > i$. In this case, the algorithm continues as before. Consequently, the total number of iterations reduces to C , and the overall running time becomes

$$O(\min(C\sqrt{n}, n + C) \cdot m).$$

Chapter 5

Necessarily Rank-Maximal Matching

Rank-Maximal matchings dealt with complete preference profiles for all agents. Necessarily Rank-Maximal Matchings work with partial preference profiles, while still finding a way to find an optimal matching assignment.

Definition 5.0.1. A matching M with respect to a partial preference profile P is said to be Necessarily Rank-Maximal if the matching is Rank-Maximal with respect to any possible extension of P to a complete preference profile that is consistent with P .

5.1 Elicitation Protocols (Query Models)

Online algorithms start with an empty preference profile i.e, no agent has revealed any of her preferences. So, it is needed to start from the very scratch and ask agents to reveal their preferences - this is querying and different query models device different types of queries(questions) to elicit the preferences. The goal is to get as much as preferences so that the algorithm can guarantee to produce an NRM from the revealed preference lists.

5.1.1 Next-best query model

An agent is asked to reveal the very next preferred object given all the higher preferences are elicited.

Query $Q(a_i, k)$ asks agent $a_i \in N$ to reveal the object in the k -th position in her preference list $\succ$, $k \in [n]$. This is a valid query only if query $Q(a_i, j)$ is taken $\forall j \in [k - 1]$.

This model builds a partial preference profile $\succ'$.

5.1.2 Hybrid-query model

Two types of queries are asked in this model.

1. **rank query** $Q(a_i, k)$: for an agent $a_i \in N$ and $k \in [n]$, the query returns the object $o_j \in O$ such that $\text{rank}(a_i, o_j) = k$.
2. **object query** $Q(a_i, o_j)$: for an agent $a_i \in N$ and an object $o_j \in O$, the query returns $\text{rank}(a_i, o_j)$.

The partial preferences are expressed as an incomplete rank' function.

5.1.3 Set-compare model

The model asks query $Q(a_i, O')$ where $a_i \in N$ is an agent and $O' \subseteq O$ is a subset of objects, and returns $\max_i(O')$, which is the object a_i likes the best in O' .

Here, each agent a_i holds a partial order¹ $\succ'_i$.

Every pair of objects in $\succ'_i$ does not need to be comparable (we might not be able to say which one is preferred) since they may be outputs of queries involving different H' . Also, we are not able to induce an incomplete rank function rank' from $\succ'_i$ because ranks of objects are never retrieved in this model.

Here, we will introduce a modified version of the Set-Compare Model called **Rank-Aware Set-Compare Model (RASC)** in chapter 7 which helps to elicit preferences efficiently. The efficiency is achieved by minimizing the number of queries needed to obtain the required number of preferences for the algorithm to compute an NRM, thus optimizing the cost of the online algorithm and improving the competitive ratio.

5.2 Checking whether a matching is NRM

The algorithm to check whether a matching is NRM makes use of another algorithm that computes the optimal signature given a subset of agents ($S \subseteq N$), a subset of objects ($T \subseteq O$) and a set of forbidden edge pairs ($F \subseteq S \times T$).

5.2.1 Helper Algorithm to find optimal matching

Input: Preference profile P , $S \subseteq N$, $T \subseteq O$, $F \subseteq S \times T$

Output: $\text{sig}_{P'}^{\text{opt}}(M)$ and $\text{argsig}_{P'}^{\text{opt}}(M)$

This algorithm finds the optimal signature associated with a matching. So, it considers the best case scenario, which is when all unrevealed objects

¹A partial order is a binary relation on a set that is reflexive, antisymmetric, and transitive. Unlike a total order, a partial order does not require every pair of elements to be comparable, meaning some pairs might be "incomparable."

of an agent a_i are ranked $k_i + 1$, i.e., the first unrevealed position for that agent. Even though all objects are supposed to have a unique rank for a particular agent, we consider all unrevealed objects for an agent to have the best possible rank in order to obtain the optimal signature. With this idea in mind, we create the (weak) preference profile P' and a graph G that follows this preference profile. Now, the rank-maximal matching of this graph G would give us the optimal signature, since this graph G embodies the best-case scenario obtainable for the given preference profile, S , T , and F .

Algorithm 4 Algorithm to compute optimal signature and the matching corresponding to the optimal signature

- 1: **for all** $a_i \in S$ **do**
 - 2: $k_i \leftarrow |P_i|$
 - 3: **end for**
 - 4: Create $P' \leftarrow$ A weak preference profile such that $\forall(a_i, o_j) \in rev(P)$, o_j is matched to the same rank in P' as it was in P and $\forall(a_i, o_j) \in unrev(P)$, o_j is matched to a_i at the $(k_i + 1)$ st position in P'
 - 5: Create a graph $G = (S \cup T, S \times T \setminus F)$ such that agents are matched to their objects as stipulated by preference profile P'
 - 6: $M \leftarrow$ Rank-Maximal matching of G
 - 7: **return** $sig_{P'}(M)$ and M
-

Time Complexity Analysis

Assume that $C = \max(k_i) \forall i \in [n]$

Calculation of k_i for all a_i can be performed in $O(nC)$. Similarly, $O(nC)$ time would be required to construct the weak preference profile and the graph. Computing the Rank-Maximal matching with the algorithm mentioned in chapter 4 would take $O(\min(C\sqrt{n}, n + C) \times m)$ time. Since $C \leq n$, the calculation of the Rank-Maximal matching would dominate the running time. Hence, the algorithm runs in $O(\min(C\sqrt{n}, n + C) \times m)$ time.

5.2.2 Algorithm to determine whether a matching is NRM

Input: Preference Profile P , N , O , M

Output: Whether the given M is an NRM

We know that a matching can be a candidate for NRM only if it has a size of at least $n - 1$. So, if $|M| \leq n - 2$, then M is definitely not an NRM. If $|M| = n$, then M must have the optimal signature. If M has a signature less than the optimal signature, then a matching with the optimal signature would be lexicographically higher and therefore more qualified to be the NRM. In this

case, if M has the optimal signature, we can immediately determine that M is NRM. On the other hand, if $|M| = n - 1$, then there exists an agent-object pair (a_i, o_j) such that it is not revealed in P . Since it is not clear whether this agent-object pair would match in the optimal matching, we need to examine two possibilities:

1. **(a_i, o_j) is present in the optimal matching.**
 In this case, we exclude a_i from N , o_j from O , and force the edge (a_i, o_j) by solving the reduced instance.
 Input instance: $(N \setminus \{a_i\}, O \setminus \{o_j\}, \phi)$
 If this gives a signature better than $sig_P(M)$, then using (a_i, o_j) gives a better matching, so M cannot be an NRM.
2. **(a_i, o_j) is absent in the optimal matching.**
 In this case, we add (a_i, o_j) to the forbidden set.
 Input instance: $(N, O, (a_i, o_j))$
 If this gives a signature better than $extsig_P(M)$, then not using (a_i, o_j) gives a better matching, so M cannot be an NRM.

If the signature of M is at least as good as the signature of the above two cases, no matching can be better than M , which means that M is the NRM.

Algorithm 5 Algorithm that returns whether a given matching M is NRM

```

1:  $size \leftarrow |M \cap rev(P)|$ 
2: if  $size = n$  and  $sig_P(M) = sig_P^{opt}$  then
3:   return Yes
4: else if  $size = n - 1$  then
5:    $(a_i, o_j) \leftarrow M \cap rev(P)$ 
6:   if  $sig_P(M) \succeq sig_P^{opt}(N \setminus \{a_i\}, O \setminus \{o_j\}, \phi)$  and  $extsig_P(M) \succeq sig_P^{opt}(N, O, (a_i, o_j))$  then
7:     return Yes
8:   end if
9: end if
10: return No

```

Time Complexity Analysis

Calculating the size would take $O(nC)$ naively since there are at most $O(nC)$ revealed pairs. Calculation of sig_P^{opt} can be done using Algorithm 3 and would take $O(\min(C\sqrt{n}, n + C) \times m)$ and $sig_P(M)$ can be calculated in $O(n)$. Thus, the running time of sig_P^{opt} would dominate and the algorithm runs in $O(\min(C\sqrt{n}, n + C) \times m)$.

5.3 Computing an NRM, if it exists

Input: Preference profile P , N , O

Output: Returns an NRM for the given input, if it exists, otherwise returns NO

The algorithm first finds M , the Rank-Maximal matching with respect to the given input. If the matching is perfect, i.e., if it has size n and also has the optimal signature, then no other matching could possibly be better, so we return M as the NRM. If not, we still hope for an NRM in case M is of size $n - 1$. (However, if M has size n but does not have the optimal signature, an NRM would not exist since there is no additional agent-object pair whose addition would make the matching better, and M already has the highest signature for a size n matching since it is rank-maximal.) In this case, we have an unrevealed agent-object pair in NRM. Since the identity of the unrevealed agent-object pair is unknown, we iterate through all agent-object pairs (a_i, o_j) and calculate the Rank-Maximal matching E , for the reduced input instance (without a_i and o_j). We then add (a_i, o_j) to E and check if this matching is NRM with respect to P . If it is, then (a_i, o_j) of that iteration is the unrevealed pair whose inclusion would give us an NRM. If no such agent-object pairs can give us an NRM by reduction of the input instance, then no NRM exists for the given input, since the rank-maximal matching should have a size of at least $n - 1$ for an NRM to exist. Thus, we return NO in this case.

Algorithm 6 Algorithm to compute an NRM, if it exists

```

1:  $M \leftarrow$  Rank-Maximal matching with respect to  $P$ , given  $N$  and  $O$ 
2: if  $|M \cap rev(P)| = n$  and  $sig_P(M) = sig_P^{opt}$  then
3:   return  $M$ 
4: else
5:   for  $(a_i, o_j) \in unrev(P)$  do
6:      $E \leftarrow$  Rank-Maximal matching of  $N \setminus \{a_i\}$  to  $O \setminus \{o_j\}$  with respect
       to  $P$ 
7:      $M' \leftarrow E \cup \{(a_i, o_j)\}$ 
8:     if  $M'$  is NRM with respect to  $P$  then
9:       return  $M'$ 
10:    end if
11:  end for
12: end if
13: return NO

```

Time Complexity Analysis

Calculating M would take $O(\min(C\sqrt{n}, n + C) \times m)$ with algorithm 3. Calculating sig_P^{opt} and checking to see if M' is an NRM at line 8 would take the same amount of time. Hence, the algorithm takes $O(\min(C\sqrt{n}, n + C) \times m)$ time since these steps dominate its running time.

5.4 Elicitation Algorithm for Necessarily Rank-Maximal Matching with Next-Best Model

Input: N, O

Output: An NRM

Algorithm 7 Elicitation algorithm for NRM

```

1:  $A \leftarrow N$ 
2:  $H \leftarrow O$ 
3:  $E \leftarrow \phi$ 
4:  $M \leftarrow \phi$ 
5:  $F \leftarrow \phi$ 
6: if  $|N| = 2$  then
7:    $h_1 = Q(a_1)$ 
8:   return  $[a_1, h_1], [a_2, h_2]$ 
9: end if
10: for  $i$  in range from 1 to  $n - 1$  do
11:   for  $a$  in  $A$  do
12:      $h \leftarrow Q(a)$ 
13:     if  $h \in H$  and  $(a, h) \notin F$  then
14:        $E = E \cup \{(a, h)\}$ 
15:     end if
16:   end for
17:   Augment  $M$  so that it is a maximum matching in  $(N \cup O, E)$ 
18:   Calculate the Edmond-Gallai Decomposition  $\mathcal{U}, \mathcal{E}, \mathcal{O}$  for  $M$ 
19:   If agent  $a \in N$  is  $\mathcal{U}$  or  $\mathcal{O}$ , remove  $a$  from  $A$ 
20:   If object  $o \in \mathcal{O}$  is  $\mathcal{U}$  or  $\mathcal{O}$ , remove  $o$  from  $H$ 
21:   Add any  $\mathcal{OO}$  or  $\mathcal{OU}$  edges to  $F$  and remove them from  $E$ 
22: end for
23: return  $M$ 

```

This algorithm queries for the next-best position of each agent only when necessary and uses Edmonds-Gallai decomposition to incrementally create an NRM from partial preferences. It stores the number of unfinished agents A , the number of available objects H , the number of edges currently revealed in the graph E , the matching M , and the set of forbidden edges F . We perform an initial check to see if the number of agents is just 2, and match a_1 with its best choice, and a_2 with the other object. If we have more than 2 agents, the algorithm iterates $n - 1$ times, iteratively improving the matching M obtained after each step so that it is an NRM after the $(n - 1)$ th step. In each iteration, it goes over each unfinished agent and asks for their next-best object. If the object is available and is not forbidden for the requesting agent, we add the agent-object edge in the edgeset of the graph. After all $n - 1$ agents are queried, we update M to be a maximum matching in $(N \cup O, E)$ by finding M -augmenting paths. We then decompose the vertices of M into E, U and O . For an agent or object

in U or O , their matched pair is fixed with respect to all future matchings, so they cannot contribute to future improvements and can be removed from A and H , respectively. We also know that OO and OU edges can not be part of any M -augmenting path and therefore cannot appear in any rank-maximal matching. Hence, any such edges, if present, can be removed from E and added to F . After $n - 1$ iterations, we would have only one unmatched agent-object pair, which can be matched directly, and thus we get our NRM.

Competitive Ratio

Let $r_a = \min_i(a \notin \mathcal{E})$

For an agent matched to an unrevealed object, we ask 0 queries in the optimal algorithm (There can be at most one agent matched to an unrevealed preference). For all other agents, the optimal algorithm must ask at least one query. Hence, for each agent, the optimal algorithm asks $\min(r_a - 1, 1)$ queries. This algorithm asks r_a queries for each agent a that is matched to its revealed preference and at most $n - 1$ queries to the agent matched to its unrevealed preference.

Let $A' \leftarrow$ set of agents matched to their revealed preference.

Hence,

$$\text{competitive ratio} = \frac{ALG}{OPT} = \frac{\sum_{a \in A'} (r_a) + n - 1}{\sum_{a \in A'} (\min(r_a - 1, 1))} = \frac{\sum_{a \in A'} (r_a + 1)}{\sum_{a \in A'} (\min(r_a - 1, 1))} \leq 3$$

Thus, the competitive ratio is upper bounded at 3. However, it can be brought down and shown that the algorithm is 3/2 competitive.

Chapter 6

Our Results

Irving et. al. [10] gave a combinatorial algorithm to find a rank-maximal matching. Later, Hosseini(2021)[7] solved the problem of checking whether a matching is necessarily a rank-maximal matching (NRM) under partial preferences (top-k model) and used this to compute an NRM from the given top-k list. He also proved that in the next best query model, no algorithm can be better than $4/3$ -competitive. Peters(2022)[13] came up with a $3/2$ -competitive online algorithm for eliciting an NRM in next-best query model addressing Hosseini’s open conjecture. He also introduced the two new query models such as the hybrid query model and set compare model. For hybrid query model, they developed a 6-competitive NRM algorithm in hybrid query model. Peters succeeded in developing NRM elicitation algorithms of constant competitive ratio except for set-compare model. He posed an open question to find an algorithm with a constant ($O(1)$) or even sublinear ($O(n^c)$ where $c < 1$) competitive ratio for eliciting an NRM in the set-compare model.

We address the open challenge and try to develop a modified version of the set-compare model to elicit NRMs efficiently.

6.1 Limitations of Set-Compare model

The following is a simple algorithm to find an NRM using the Set-Compare Model. $S[i]$ contains the objects to be queried for a_i . This algorithm queries for the set-compare position of each agent and makes use of Edmonds-Gallai decomposition to incrementally create an NRM from partial preferences. It stores the number of unfinished agents A , the number of available objects H , the number of edges currently revealed in the graph E , the matching M , and the set of forbidden edges F . We perform an initial check to see if the number of agents is just 2, and match a_1 with its best choice, and a_2 with the other object. If we have more than 2 agents, the algorithm iterates $n - 1$ times, iteratively improving the matching M obtained after each step so that it is an NRM after the $(n - 1)^{th}$ step. In each iteration, it goes over each unfinished agent and

asks for their best object in the set $S[i]$. $S[i]$ would have all the objects except for those already revealed for that particular agent. If the object is available and is not forbidden for the requesting agent, we add the agent-object edge in the edgeset of the graph. After all $n - 1$ agents are queried, we update M to be a maximum matching in $(N \cup O, E)$ by finding M -augmenting paths. We then decompose the vertices of M into $\mathcal{E}, \mathcal{U}$ and $\mathcal{O}$. For an agent or object in $\mathcal{U}$ or $\mathcal{O}$, their matched pair is fixed with respect to all future matchings, so they cannot contribute to future improvements and can be removed from A and H , respectively. We also know that $\mathcal{OO}$ and $\mathcal{OU}$ edges can not be part of any M -augmenting path and therefore cannot appear in any rank-maximal matching. Hence, any such edges, if present, can be removed from E and added to F . After $n - 1$ iterations, we would have only one unmatched agent-object pair, which can be matched directly, and thus we get our NRM.

Algorithm 8 Elicitation algorithm for NRM with Set-Compare Model

```

1:  $A \leftarrow N$ 
2:  $H \leftarrow O$ 
3:  $E \leftarrow \phi$ 
4:  $M \leftarrow \phi$ 
5:  $F \leftarrow \phi$ 
6: if  $|N| = 2$  then
7:    $h_1 = Q(a_1, S[1])$ 
8:   return  $[a_1, h_1], [a_2, h_2]$ 
9: end if
10:  $S = [H] * n$ 
11: for  $i$  in range from 1 to  $n - 1$  do
12:   for  $a_i$  in  $A$  do
13:      $h \leftarrow Q(a_i, S[i])$ 
14:     Remove  $h$  from  $S[i]$  ▷ We remove revealed object
15:     if  $h \in H$  and  $(a_i, h) \notin F$  then
16:        $E = E \cup \{(a_i, h)\}$ 
17:     end if
18:   end for
19:   Augment  $M$  so that it is a maximum matching in  $(N \cup O, E)$ 
20:   Calculate the Edmund-Gallai Decomposition  $\mathcal{U}, \mathcal{E}, \mathcal{O}$  for  $M$ 
21:   If agent  $a \in N$  is  $\mathcal{U}$  or  $\mathcal{O}$ , remove  $a$  from  $A$ 
22:   If object  $o \in O$  is  $\mathcal{U}$  or  $\mathcal{O}$ , remove  $o$  from  $H$ 
23:   Add any  $\mathcal{OO}$  or  $\mathcal{OU}$  edges to  $F$  and remove them from  $E$ 
24: end for
25: return  $M$ 

```

We see that, after each iteration, the only objects omitted from the set to be queried for each agent are the ones already revealed by querying. In other words, we have to query the object at each position for each agent since we would not be able to ascertain the rank of the most preferred object given by

the Set-Compare model in any other way. This reduces to the next-best query model, since we query position by position for each agent. Thus, algorithm 7 is equivalent to algorithm 6 given in the previous chapter.

6.2 Modifying Set-Compare model

We have seen that the vanilla set-compare model will not suffice to elicit preferences efficiently and collapses to the next-best query model at the end of the day but with less efficiency.

Here we make an attempt to modify the vanilla set compare model. The plan is to reduce the size of the subset of objects used in each query. Let us see what happens in this approach to optimization. Let us look into a simple 3-agents-3-objects system in which we start with the empty preference lists of the agents. The modified set-compare model asks each agent to pick their best choice from a subset of objects. **But unlike the classic version, the size of the agent-specific subset is reduced in each iteration by removing the odd, unmatched and agent-revealed even objects from the previous one.** Consider two complete preference profiles ($\succ_1$ and $\succ_2$)(note that the ranks of all (a_i, o_j) pairs remain hidden at start). Let us try to elicit the preferences of each agent. The results of the following steps exactly coincide in the case of both of the above preference profiles, hence written once only.

Let E be the set of all revealed edges and M be a maximum matching in the graph $G = (N, O)$. A_a is the set of all unmatched agents and A_o is the set of all available objects. Initially, the subset O' in $Q(a_i, O')$ is the set of all objects itself. We use a set of all objects available for querying.

$$O' \leftarrow O$$

$$E \leftarrow \phi, M \leftarrow \phi$$

$$A_a \leftarrow \phi, A_o \leftarrow \phi$$

$$Q(a_1, O') = o_1$$

$$Q(a_2, O') = o_1$$

$$Q(a_3, O') = o_2$$

$$\succ: \quad a_1 : o_1$$

$$a_2 : o_1$$

$$a_3 : o_2$$

$$M = \{(a_1, o_1), (a_3, o_2)\}$$

Now apply the Edmonds-Gallai decomposition (4.2.2)

$$O = \{o_1\}$$

$$E = \{a_1, a_2, o_3\}$$

$$U = \{a_1, o_2\}$$

Delete all vertices that fell into $U \cup O$ from A_a and A_o .

$$A_a \leftarrow A_a \cap E \Rightarrow A_a = \{a_1, a_2\}$$

$$A_o \leftarrow A_o \cap E \Rightarrow A_o = \{o_3\}$$

Observe that the query subset (to be used in the next iteration) has been

reduced to the singleton set of o_3 since both o_1 and o_2 became odd or unreachable and will not be matched to an edge of lower preference.

$$O' \leftarrow A_o \Rightarrow O' = \{o_3\}$$

Now query only unmatched agents using available objects.

$$Q(a_1, O') = o_3$$

$$Q(a_2, O') = o_3$$

$$O = \{o_1\}$$

$$E = \{a_1, a_2, o_3\}$$

$$U = \{a_1, o_2\}$$

The following two preference profiles can be created from the above information.

$$\begin{array}{ll} \succ' & a_1 : o_1 > o_2 > o_3 \\ & a_2 : o_1 > o_3 > o_2 \\ & a_3 : o_2 > o_3 > o_1 \\ \succ'' & a_1 : o_1 > o_3 > o_2 \\ & a_2 : o_1 > o_2 > o_3 \\ & a_3 : o_2 > o_3 > o_1 \end{array}$$

Moreover, a rank-maximal matching for the instance $\succ'$ is not a rank-maximal matching in $\succ''$ and vice versa. Because this type of elicitation does not give us any information on the position of the o_3 in the list of the agents. The failure in the above example where the only difference between $\succ'$ and $\succ''$ was the change in the positions of o_2 and o_3 in the list of both a_1 and a_2 , is a direct consequence of this.

$$\text{In } \succ', \quad a_1 : o_1 > \mathbf{o}_3 > \mathbf{o}_2$$

$$\text{In } \succ'', \quad a_1 : o_1 > \mathbf{o}_2 > \mathbf{o}_3$$

The information collected by this query model is not sufficient for the algorithm to compute an optimal matching. Let us modify the set-compare protocol and introduce a new elicitation protocol called rank-aware set-compare model. We propose an algorithm to elicit NRM based on the same.

6.3 Rank-Aware Set-Compare Model

The model asks an agent to select their most preferred object from a given subset of objects and also retrieves the rank of the selected object in her preference list. Formally,

$$Q(a_i, O') = (o, k), \quad O \subseteq O'$$

where

$$o = \arg \max_{o' \in O'} \text{rank}(a_i, o')$$

$$k = \text{rank}(a_i, o),$$

with respect to $\succ_i$, the complete preference list of agent a_i .

Each query in the new protocol retrieves more information from the agent than the vanilla set-compare model. Let us check how the new model improves upon the existing set-compare model. **Once an object reveals its best choice and the rank of the corresponding object from the given post, the model is able to remember this (object, rank) and is intelligent enough to avoid making queries to the particular agent till the iteration which is greater than the revealed rank.** Now we propose an algorithm based on this new query model.

Algorithm 9 Elicitation algorithm for NRM with Rank-Aware Set-Compare Model.

Input: N, O

Output: An NRM

```

1:  $A \leftarrow N$   $\triangleright A$  is the set of unfinished agents
2:  $H \leftarrow [O] * n$   $\triangleright H$  is a set of sets where  $H_i$  is the list of available objects for  $a_i$ 
3:  $E \leftarrow \phi$ 
4:  $M \leftarrow \phi$ 
5:  $F \leftarrow \phi$ 
6:  $D \leftarrow \{\}$   $\triangleright$  Contents of  $D \rightarrow agent : (rank, object)$ 
7: if  $|N| = 2$  then
8:    $h_1 = Q(a_1, H_1)$ 
9:   return  $[a_1, h_1], [a_2, h_2]$ 
10: end if
11: for  $i$  in range from 1 to  $n - 1$  do
12:   for  $a_i$  in  $A$  do
13:      $skip \leftarrow False$ 
14:      $h \leftarrow None$ 
15:      $r \leftarrow None$ 
16:     if  $a_i$  in  $keys(D)$  then
17:        $retrievedRank, retrievedObj \leftarrow D[a_i]$ 
18:       if  $retrievedRank > i$  then
19:          $skip \leftarrow True$ 
20:       else if  $i = retrievedRank$  then
21:          $h \leftarrow retrievedObj$ 
22:          $r \leftarrow retrievedRank$ 
23:          $skip \leftarrow True$ 
24:         Remove  $a_i$  from  $D$ 
25:       end if
26:     end if
27:     if  $skip = False$  then
28:        $h, r \leftarrow Q(a_i, H_i)$ 
29:       Remove  $h$  from  $H_i$ 
30:     end if
31:     if  $h$  is not  $None$  and  $r$  is not  $None$  then
32:       if  $(a_i, h) \notin F$  and  $r = i$  and  $h \in H_i$  then
33:          $E = E \cup \{(a_i, h)\}$ 
34:       else if  $r > i$  then
35:          $D[a_i] = (r, h)$ 
36:       end if
37:     end if
38:   end for
39:   Augment  $M$  so that it is a maximum matching in  $(N \cup O, E)$ 
40:   Calculate the Edmond-Gallai Decomposition  $\mathcal{U}, \mathcal{E}, \mathcal{O}$  for  $M$ 
41:   If agent  $a \in N$  is  $\mathcal{U}$  or  $\mathcal{O}$ , remove  $a$  from  $A$ 
42:   If object  $o \in O$  is  $\mathcal{U}$  or  $\mathcal{O}$ , remove  $o$  from  $H_i \forall i \in [n]$ 
43:   Add any  $\mathcal{OO}$  or  $\mathcal{OU}$  edges to  $F$  and remove them from  $E$ 
44: end for
45: return  $M$ 

```

The algorithm stores the number of active agents A , the number of available objects H , the number of edges currently revealed (and added in the graph) E , the matching M and the set of forbidden edges F . We perform an initial check to see if the number of agents is just 2, and match a_1 with its best choice, and a_2 with the other object. If we have more than 2 agents, the algorithm iterates $n - 1$ times, iteratively improving the matching M obtained after each step so that it is an NRM after the $(n - 1)^{th}$ step.

In each iteration, we use the newly introduced RASC model to obtain preferences from the agents. We use a dictionary with agents as key and an ordered pair $(rank, object)$ as value. The ordered pairs stored are the results of querying that may be useful in future iterations. Thus, this agent needs to reveal her next choice only when the iteration count exceeds this $rank$. This additional memory helps the algorithm decide whether an agent is to be queried in a particular iteration, significantly reducing the total number of queries.

We iterate through each available agent a_i in each iteration. We first see if there is a need to query a_i or not. For this, we introduce the boolean variable *skip*, which is initially set to *False*. If we find a_i in the dictionary D , then it means that there is a possibility that the's preferred object of a_i for the current iteration is stored in D . If rank happens to be the current iteration number, we have the rank and object for a_i for the current iteration, that is, $(rank, object)$ tuple in $D[a_i]$, and we can thus skip the current query and use *object* from the tuple and set the value of the key a_i as null. In this case, we add the edge (a, h) to E if it is not in the forbidden set (F) . On the other hand, if the rank is higher, we skip the current query for a_i . If the key a_i is already null in the dictionary, we query a_i and remove the retrieved object from the available object set H_i of a_i and update the dictionary with the new (r, o) for the key a_i .

Suppose some query gave $Q(a_i, H_i) = (r, o)$, where H_i is the agent-specific subset of objects.

This information is stored in dictionary D as a key-value pair $a_i : (r, o)$

$D[a_i] = (r, o)$

we let a_i retire till iteration $n = r$ using $O(1)$ look-ups ($D(a_i)$) checking the rank of the current best revealed object of the agent against the iteration number i .

The next query on a_i is made only when $i = r + 1$.

In iteration $i = r$, retrieve the object o stored in the dictionary for a_i , add the edge $(a_i, D(a_i).first) = (a_i, o)$ to the graph if not forbidden.

$D(a_i) = null$ (**remember to flush the values of a_i stored in the dictionary once we use it. The key a_i can be flushed too, since it's value is empty**).

In iteration $i = r + 1$, our a_i key is null, forcing us to query a_i

store the tuple in the dictionary

delete the new object from the agent-specific subset a_i (it should not appear as

a candidate for H_i even if it remains as even).

We then proceed to find a maximum matching, apply the Edmond-Gallai decomposition, and remove the unwanted edges and occupied vertices.

All queries to a_i between the iteration in which it revealed (r, o) and the currently revealed rank r are futile since those objects in the preference list of a_i are inactive (already odd or unreachable). **We have avoided those queries using the rank information revealed along with the best choice of the last query.** The correctness of the rest of the algorithm follows from the correctness of algorithm 7[?].

We are planning to do the competitive analysis of this online algorithm during next semester.

6.4 Analysis of the Algorithm

Definition 6.4.1. [5] Let $G = (A \cup P, E)$ be a bipartite graph with ranks on the edges belonging to $\{1, 2, \dots, r\}$. Suppose that $a \in A, p \in P$ and (a, p) does not belong to E . Let $H = (A \cup P, E \cup \{(a, p)\})$. We denote a critical rank of (a, p) in H as follows.

Lemma 2. [5] Let G, H and (a, p) be as in Definition 4. Then the critical rank of (a, p) is c if and only if

- For every $1 \leq i < c$, the edge (a, p) belongs to every rank-maximal matching of H_i , in which (a, p) has rank i , and
- For every $c < i \leq r$, the edge (a, p) does not belong to any rank-maximal matching of H_i , in which (a, p) has rank i , and
- There exists a rank-maximal matching M of H_c , in which (a, p) has rank c such that (a, p) is not contained in M .

Lemma 3. If we swap positions of two posts p_i and p_j both belonging to r_a , where $\text{rank}(a, p_i) < \text{rank}(p_j)$ in the preference list of a , then after the swap a will be matched to p_j in every RMM. Here, r_a denotes the posts that are revealed by the algorithm for an agent a .

Proof. Since p_j gets queried after p_i is processed, critical rank of $(a, p_j) \geq \text{rank}(a, p_i) + 1$.

On swapping, p_j will be at the position $\text{rank}(a, p_i) \leq$ critical rank of $(a, p_j) - 1$ i.e., $\text{rank}(a, p_i) < \text{critical rank of } (a, p_j)$. Therefore, by lemma 2, p_j will be present in every rank maximal matching. $\square$

Lemma 4. The optimal algorithm must match every agent except at most 1 to a revealed post.

Proof. [7] The optimal algorithm returns a matching M which is an NRM. We know that all NRMs are also NPOs (Necessarily Pareto Optimal Matching). Now, consider the case where optimal algorithm is matched to 2 agents which are unrevealed. Let them be a_1 and a_2 matched to unrevealed objects o_1 and o_2 respectively. Since they are unrevealed, we can make a completion of the preference lists of a_1 and a_2 so that a_1 places o_2 before o_1 and a_2 places p_1 before o_2 in their respective preference lists.

Consider a matching M' where all agents a' in $A' = A \setminus \{a_1, a_2\}$ are matched to $M(a')$ and a_1 and a_2 are matched to o_2 and o_1 respectively. This matching makes a_1 and a_2 better off and since no one else is worse off, M' pareto dominates M , contradicting the fact that M is an NPO. Since M is not an NPO, it is not an NRM either. Therefore, by contradiction we achieve that the optimal algorithm must match at least $n - 1$ agents to their revealed preferences. $\square$

Lemma 5. *Optimum must reveal at least $q_a - 2$ objects for an agent a that is revealed, where q_a denotes the number of agents revealed by the algorithm for a .*

Proof. Let us consider a case where the optimal has revealed only $q_a - 3$ objects for an agent a .

Consider a general case where o_i is queried at iteration $i' \leq i$, o_j at iteration $j' \leq j$ and o_k at iteration $k' \leq k$. We divide this into various cases depending on the object a is matched to in the NRM. We utilize lemma 3 in these cases.

- If a is matched to a revealed object, critical rank of $(a, o_i) < \text{rank}(a, o_j)$ so o_i and o_j can be swapped. By lemma 3, o_j must be matched in every rank-maximal matching and therefore the NRM of the preference profile.
- If a is matched to o_i , critical rank of $(a, o_i) < \text{rank}(a, o_j)$ so o_i and o_j can be swapped to make o_j matched to a in the NRM.
- If a is matched to o_j , critical rank of $(a, o_j) < \text{rank}(a, o_k)$ so o_k and o_j can be swapped to make o_k matched to a in the NRM.
- If a is matched to o_k , critical rank of $(a, o_i) < \text{rank}(a, o_j)$ so o_i and o_j can be swapped to make o_j matched to a in the NRM.

In all cases, we prove that the agent matched to a in the NRM is different to that returned by the algorithm, thus proving that optimal is wrong in such a scenario.

This contradicts our assumption that optimal algorithm can work with $q_a - 3$ revealed agents. $\square$

Lemma 6. *The algorithm is a 4-competitive algorithm for eliciting a rank-maximal matching in the RASC query model.*

Proof. We calculate the competitive ratio:

$$\alpha = \frac{ALG}{OPT} \leq \frac{\sum_{a \in A'} q_a + n - 1}{\sum_{a \in A'} \max(1, q_a - 2)} = \frac{\sum_{a \in A'} q_a + 1}{\sum_{a \in A'} \max(1, q_a - 2)}$$

Here, A' denotes the set of all agents that are matched to revealed objects.

All agents in A' are asked q_a queries by the algorithm as per the definition of q_a . The last agent which might or might not be matched to an unrevealed agent will ask a maximum of $n - 1$ queries.

Optimum must ask atleast $q_a - 2$ queries for an agent a by lemma 5. However, in case of $q_a \leq 2$, optimal can not finalize with 0 queries asked, hence we assign 1 query to such a case for the optimal algorithm.

The expression for α returns the maximum value of 4 for $q_a = 3$. Thus,

we conclude that our algorithm gives an upper bound of 4 on the competitive ratio. $\square$

Bibliography

- [1] David J. Abraham, Robert W. Irving, and Telikepalli Kavitha. Popular matchings. In *Proceedings of the 14th Annual European Symposium on Algorithms (ESA)*, volume 4168 of *Lecture Notes in Computer Science*, pages 20–31. Springer, 2006.
- [2] Haris Aziz, Serge Gaspers, Simon Mackenzie, and Toby Walsh. The logic of necessarily stable matchings under partial preferences. *Artificial Intelligence*, 277:103172, 2019.
- [3] Evripidis Bampis, Bruno Escoffier, Christine Gaspin, and Rémi Tourniaire. Query-efficient algorithms for stable matchings under uncertain preferences. In *Proceedings of the 32nd International Joint Conference on Artificial Intelligence (IJCAI)*, pages 62–68. IJCAI, 2023.
- [4] David Gale and Lloyd S. Shapley. College admissions and the stability of marriage. *The American Mathematical Monthly*, 69(1):9–15, 1962.
- [5] Pratik Ghosal and Katarzyna Paluch. Manipulation strategies for the rank maximal matching problem. October 2017.
- [6] Dan Gusfield and Robert W. Irving. *The Stable Marriage Problem: Structure and Algorithms*. MIT Press, 1989.
- [7] Hadi Hosseini, Vijay Krishna Menon, Nisarg Shah, and Sujoy Sikdar. Necessarily optimal one-sided matchings. In *Proceedings of the 30th International Joint Conference on Artificial Intelligence (IJCAI)*, pages 245–251. IJCAI, 2021.
- [8] Robert W. Irving. Stable marriage and indifference. *Discrete Applied Mathematics*, 48(3):261–272, 1994.
- [9] Robert W. Irving, Telikepalli Kavitha, Kurt Mehlhorn, Dimitrios Michail, and Katarzyna Paluch. Rank-maximal matchings. *ACM Transactions on Algorithms*, 2(4):602–610, 2006.
- [10] Robert W. Irving, Telikepalli Kavitha, Kurt Mehlhorn, Dimitrios Michail, and Katarzyna E. Paluch. Rank-maximal matchings. In J. Ian Munro, editor, *Proceedings of the Fifteenth Annual ACM-SIAM Symposium on Dis-*

crete Algorithms, SODA 2004, New Orleans, Louisiana, USA, January 11-14, 2004, pages 68–75. SIAM, 2004.

- [11] Telikepalli Kavitha. Fair matchings. *SIAM Journal on Computing*, 51(4):933–964, 2022.
- [12] Meena Mahajan. Matchings in graphs. Lecture notes for the course on Graph Theory, January 2010. Lecture notes by Rajesh Chitnis, Meeting 1.
- [13] Jannik Peters. Online elicitation of necessarily optimal matchings. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 5164–5172, 2022.
- [14] Alvin E. Roth. The evolution of the labor market for medical interns and residents: A case study in game theory. *Journal of Political Economy*, 92(6):991–1016, 1984.
- [15] Chung Piaw Teo and Jay Sethuraman. Substitutes, complements, and stable matching with incomplete preferences. *Operations Research Letters*, 22(3–5):165–172, 1998.